VOTECHAIN: END-TO-END DATA TRANSFER, SECURITY & TIMING

Complete Technical Pipeline with Step-by-Step Info & Real-World Walkthrough Example

System Overview: In the VoteChain Biometric Web3 Voting System, voting is executed across a coordinated four-stage hardware-software pipeline: **(1) R307 Optical Biometric Sensing**, **(2) ESP32 Edge Gateway**, **(3) Backend Recognition Engine & Relayer**, and **(4) Dual Storage: Blockchain Consensus + MongoDB Atlas**. At every step, data changes format—from analog optical reflections into high-entropy minutiae vectors, serial binary packet frames, JSON HTTP payloads, salted SHA-256 anonymized digests, and finally ECDSA-signed Ethereum smart contract transactions. Below is the complete flow diagram along with an exact step-by-step real-world walkthrough of citizen **Vallabh Joshi** voting for **BJP**.



■ Real Example Voter: Vallabh Joshi (FP_UID_1)	■■ Selected Party: BJP [Konkan - Mumbai]	■ Record Time: 2026-10-30 10:28:22 IST	■■ Sepolia Block: #12024283 (Confirmed)

CONCRETE REAL-WORLD EXAMPLE: CITIZEN VALLABH JOSHI VOTING FOR BJP

To understand the system clearly, here is the complete trace of a real voter from touch to blockchain mining:

Stage & Action	What Actually Happens In This Step	Data Form & Values in This Step	Latency
Phase A: Biometric Scan	Citizen Vallabh Joshi approaches the polling booth in Mumbai (Konkan region). He places his right index finger on the R307 sensor glass. The sensor illuminates the finger with red LEDs, captures ridges and valleys, extracts 48 minutiae points into volatile RAM, and streams 512 bytes over UART serial pins to the ESP32.	• Raw 256x288 pixel optical reflection • Minutiae Vector: 512 bytes binary • UART Frames: 4x 139-byte packets • Transferred to ESP32 Serial2 @ 57600 baud	**240 ms (0.24 s)**
Phase B: Cloud Engine Auth	ESP32 sends the minutiae hex string via Wi-Fi to Node.js /api/fingerprints/engine/verify. The server's Fingerprint Engine executes a 1:N spatial pattern search across MongoDB Atlas. It matches Vallabh's template (Slot #1, score 98%). The server checks MongoDB hasVoted and Sepolia contract. Both confirm he has NOT voted yet. Server issues a signed single-use authorization token valid for 60 seconds.	• Payload: JSON with 1024 hex chars • Match Score: 98% (High confidence) • Double-Vote Preflight: hasVoted == false • Auth Token: AUTH_F7629067FC337A4F3548... • Expiration: Strict 60 seconds	**75 ms (0.08 s)**
Phase C: Ballot Touch & Signing	ESP32 LCD displays 'Authorized! Vallabh Joshi' and triggers a welcoming audio tone. Vallabh touches Button 1 to vote for BJP. ESP32 sends the vote payload with the authorization token to /api/blockchain/vote. Node.js relayer verifies the token, hashes his ID with dynamic election salt (voterHash), and signs the castVote(voterHash, 'BJP') smart contract call.	• LCD: 'Authorized! Vallabh Joshi' • Touch Input: GPIO 13 Active Pulse • voterHash: b36bd0d7bee3670c... (SHA-256) • ABI: castVote(voterHash, 'BJP') • Signed with Relayer Secp256k1 key	**2.5 s (Human tap)**
Phase D: Settlement & Receipt	The signed transaction is broadcast to the Ethereum Sepolia network. Proof-of-Stake validators mint the vote into an immutable block (#12024283). Simultaneously, MongoDB stores the vote in votes_konkan, marks hasVoted = true in voters_konkan and fingerprintvoters, and logs the event with timestamp '2026-10-30 10:28:22 IST'. Terminal LCD displays 'Vote Recorded!' and plays 3 confirmation chimes.	• Sepolia Tx: 0x6d6b7b78757aba4c... • Block: #12024283 (PoS Consensus) • MongoDB: Saved in votes_konkan • Time: 2026-10-30 10:28:22 IST • hasVoted set to true (Prevents double vote)	**12 s block (Relay: 45ms)**

DETAILED STEP-BY-STEP TECHNICAL NOTES (WHAT HAPPENS IN EACH STEP)

Complete technical description of data format, transfer method, security enforcement, and latency for every step:

STEP 1: Physical Finger Placement & Optical Image Capture

■■ ~ 150 ms (0.15 seconds)

■■ **What Happens:** What Happens: Citizen places finger onto optical prism. Red LEDs light up the finger surface. Ridges touch the glass while valleys leave air gaps, causing light to reflect differently onto a CMOS sensor. The sensor captures an uncompressed 256x288 pixel grayscale image into temporary SRAM.

■ **Data Transferred:** Reflected optical photons -> 8-bit digital grayscale pixel matrix (256 x 288 resolution, 73,728 bytes).

■ **Form / Format:** Analog light reflection -> 8-bit Digital Pixel Array.

■ **Method / Protocol:** Total Internal Reflection (TIR) optical CMOS sensor pipeline -> Internal ImageBuffer.

■■ **Security Applied:** Physical anti-tamper optical prism surface. Direct volatile capture into sensor ImageBuffer RAM. Never written to non-volatile flash storage.

STEP 2: In-Sensor DSP Minutiae Extraction

■■ ~ 90 - 110 ms (0.10 seconds)

■■ **What Happens:** What Happens: The R307 sensor's internal DSP chip runs an edge detection and skeletonization algorithm. It thins the ridges to 1-pixel width, identifies coordinates where ridges end or bifurcate (split), and determines the angle theta. This mathematical vector is stored in CharBuffer1 in volatile RAM.

■ **Data Transferred:** ImageBuffer pixel matrix is converted into core minutiae points: ridge endings, bifurcations, orientation angles, and singular core points.

■ **Form / Format:** AS608 binary feature vector character buffer (512 bytes raw minutiae).

■ **Method / Protocol:** Internal high-speed DSP bus command 0x02 (PS_Image2Tz to CharBuffer1).

■■ **Security Applied:** One-way biometric abstraction: mathematical minutiae coordinates cannot be reverse-engineered back into an image. Volatile CharBuffer1 RAM only.

STEP 3: Biometric Minutiae Transfer: R307 Sensor -> ESP32

■■ ~ 85 - 95 ms (0.09 seconds)

■■ **What Happens:** What Happens: ESP32 sends command 0x08 (PS_UpChar). The sensor divides the 512-byte minutiae buffer into 4 packets of 128 bytes each. Each packet is wrapped with header bytes and a 16-bit arithmetic checksum. ESP32 receives them over Hardware Serial2 and verifies the checksum of each packet.

■ **Data Transferred:** 512 bytes of minutiae data divided into 4 sequential data packets of 128 bytes each, framed with headers and checksums.

■ **Form / Format:** UART Binary Packet Frames: [0xEF 0x01 | Address (4B) | Packet ID (1B) | Length (2B) | 128 Data Bytes | 16-bit Checksum] (139B per packet x 4 = 556B total).

■ **Method / Protocol:** Hardware Serial UART (Serial2: TX=GPIO 17, RX=GPIO 16) at 57,600 baud rate (8N1). Command: AS608 PS_UpChar (0x08).

■■ **Security Applied:** Point-to-point physical wired UART bus (immune to radio eavesdropping). Arithmetic 16-bit packet checksum verification on each packet prevents bit flips.

STEP 4: Edge -> Backend API Biometric Verification Request

■■ ~ 35 - 60 ms (0.05 seconds)

■■ **What Happens:** What Happens: ESP32 formats the 512 binary bytes into a 1024-character hexadecimal string. It creates an HTTP POST request to /api/fingerprints/engine/verify over Wi-Fi. The request body includes terminalId and the hex string.

■ **Data Transferred:** JSON verification payload containing terminal identifier and 1024-character hexadecimal minutiae string.

■ **Form / Format:** JSON-formatted string: { "terminalId": "ESP32_BOOTH_01", "templateData": "EF01FFFF..." } (~1.2 KB payload).

■ **Method / Protocol:** Wi-Fi IEEE 802.11 b/g/n wireless link -> TCP/IP -> HTTP POST /api/fingerprints/engine/verify (TLS 1.3 supported).

■■ **Security Applied:** WPA2-AES / WPA3 Wi-Fi radio encryption. Strict timeout bounds (4-second watchdog) prevent hanging threads or replay interception.

STEP 5: Backend Fingerprint Engine 1:N Biometric Matching

■■ ~ 20 - 40 ms (0.03 seconds)

■■ **What Happens:** What Happens: Node.js server loads the candidate templates from MongoDB Atlas into memory. The Fingerprint Recognition Engine aligns the probe minutiae points against candidate templates, calculating spatial distance (dr <= 18px) and angular alignment (dtheta <= 30 deg). If score >= 40%, it confirms a match.

■ **Data Transferred:** Incoming probe template compared against N citizen templates retrieved from MongoDB Atlas voter collections.

■ **Form / Format:** In-memory binary byte array and structured minutiae descriptor tables: (x, y, theta, type, quality).

■ **Method / Protocol:** Node.js Fingerprint Recognition Engine executing spatial point pattern matching and descriptor cross-correlation in memory.

■■ **Security Applied:** Bypasses sensor flash memory completely. Employs spatial distance and angular tolerance gates, preventing spoof attacks.

STEP 6: Voting Status Pre-Flight Check (Double-Voting Guard)

■■ ~ 80 - 180 ms (0.13 seconds)

■■ **What Happens:** What Happens: Once the voter is identified (e.g. Vallabh Joshi, FP_UID_1), the server checks if he already voted. It inspects the hasVoted boolean flag in MongoDB Atlas. Then it hashes his ID with the active round salt and queries the Ethereum Sepolia smart contract function hasVoted(voterHash).

■ **Data Transferred:** Citizen Voter ID (FP_UID_1) converted into salted hash SHA-256(voterID + activeSalt) and checked against ledger.

■ **Form / Format:** SHA-256 Hex Hash (64 chars) + JSON-RPC eth_call payload to Sepolia Ethereum node.

■ **Method / Protocol:** MongoDB Mongoose lookup + HTTPS JSON-RPC query (eth_call) to Sepolia RPC (ethereum-sepolia-rpc.publicnode.com).

■■ **Security Applied:** Anonymized identity: Citizen raw Voter ID is never sent to public blockchain. Dual pre-flight: checks both MongoDB hasVoted flag and smart contract.

STEP 7: Authorization Token Issuance: Backend -> ESP32

■■ ~ 25 - 40 ms (0.03 seconds)

■■ **What Happens:** What Happens: If voter is eligible (has not voted), the server generates a single-use authorization token using HMAC-SHA256, binds it to the voter's identity and current election round, sets a 60-second expiration, and returns HTTP 200 to the ESP32.

■ **Data Transferred:** Authorization receipt with citizen name, franchise region, allowed parties, and single-use cryptographic token.

■ **Form / Format:** JSON Payload: { "authorized": true, "voter": { "name": "Vallabh", "region": "Konkan" }, "authorizationToken": "AUTH_872A..." } (~350B).

■ **Method / Protocol:** HTTP 200 OK Response over Wi-Fi 802.11n TCP/IP link.

■■ **Security Applied:** HMAC-SHA256 signature binding voter ID, round, and nonce. Strict 60-second expiration. Single-use: revoked immediately upon voting.

STEP 8: Voter Balloting (Party Selection on Terminal)

■■ ~ 2.0 - 5.0 seconds (Human reaction time)

■■ **What Happens:** What Happens: ESP32 receives the token, clears the display, shows 'Authorized! Vallabh Joshi', and triggers a happy chime. The voter taps one of 4 capacitive touch buttons (1:BJP, 2:UBT, 3:Congress, 4:AIMIM). Hardware debounce filters noise, and the chosen party is locked in.

■ **Data Transferred:** Capacitive finger touch triggers digital pulse, identifying selected candidate party (BJP, UBT, Congress, or AIMIM).

■ **Form / Format:** Analog human capacitance -> Digital HIGH/LOW voltage pulse (3.3V) across ESP32 GPIO pins (13, 14, 25, 26).

■ **Method / Protocol:** Hardware GPIO interrupt / polling loop with 50 ms software debounce filter.

■■ **Security Applied:** 25-second countdown timer: session aborts if voter takes too long. Mutual exclusion lock: exactly one party tap is accepted per cycle.

STEP 9: Vote Dispatch: ESP32 Terminal -> Node.js Web3 Relayer

■■ ~ 30 - 50 ms (0.04 seconds)

■■ **What Happens:** What Happens: ESP32 packages voterID, party, region, district, and authorizationToken into JSON and transmits an HTTP POST to /api/blockchain/vote over Wi-Fi. The backend verifies the token and confirms the party is valid for the citizen's region.

■ **Data Transferred:** Citizen ID, selected party, registered constituency, and authorization token.

■ **Form / Format:** JSON Payload: { "voterID": "FP_UID_1", "party": "BJP", "region": "Konkan", "authorizationToken": "AUTH_..." } (~280B).

■ **Method / Protocol:** Wi-Fi TCP/IP HTTP POST /api/blockchain/vote (Port 3000).

■■ **Security Applied:** Authorization token verified before accepting payload. Regional franchise check: rejects votes for parties outside voter's registered region.

STEP 10: Cryptographic Anonymization & Transaction Signing

■■ ~ 15 - 25 ms (0.02 seconds)

■■ **What Happens:** What Happens: The Node.js Web3 Relayer strips the citizen's personal name and Voter ID. It calculates voterHash = SHA-256(voterID + activeSalt). It prepares the Ethereum transaction calling smart contract castVote(voterHash, 'BJP') and signs it with the relayer's Secp256k1 private key.

■ **Data Transferred:** voterHash = SHA-256(voterID + activeSalt). Smart contract invocation: castVote(voterHash, "BJP").

■ **Form / Format:** ABI-encoded Ethereum transaction payload signed with relayer's Secp256k1 private key (~180 bytes raw hex).

■ **Method / Protocol:** Internal Ethers.js v6 runtime + Keccak-256 + ECDSA signature (secp256k1 curve).

■■ **Security Applied:** Decoupled Zero-Knowledge privacy: Citizen identity is permanently unlinked from ballot. Dynamic salt prevents rainbow tables. Nonce queue prevents collisions.

STEP 11: Blockchain Broadcast & Consensus Confirmation

■■ Relay acceptance: ~ 50 ms | Block Mining: ~ 12 - 15 s (Simulation: 10 ms)

■■ **What Happens:** What Happens: The relayer transmits the signed transaction to Ethereum Sepolia via JSON-RPC. Global Proof-of-Stake validator nodes order the transaction into the mempool and mint it into a block. The vote is now permanently immutable.

■ **Data Transferred:** Signed raw transaction hex broadcasted to mempool, validated by Sepolia Proof-of-Stake nodes, and confirmed in block.

■ **Form / Format:** JSON-RPC eth_sendRawTransaction hex string -> Ethereum Sepolia block header & transaction receipt.

■ **Method / Protocol:** Ethereum JSON-RPC over HTTPS (TLS 1.3) to Sepolia RPC validator network.

■■ **Security Applied:** Decentralized consensus: Proof-of-Stake validator consensus ensures 100% immutability. Irreversible, publicly verifiable on Etherscan.

STEP 12: Dual-Storage Commit & Audit Trail Logging

■■ ~ 25 - 45 ms (0.03 seconds)

■■ **What Happens:** What Happens: Node.js records the cast vote in the regional MongoDB collection (votes_konkan) with party, txHash, and blockNumber. It updates hasVoted = true on the voter's record in voters_konkan and fingerprintvoters, and creates an audit log entry with an Indian Standard Time (IST) timestamp.

■ **Data Transferred:** Transaction hash, block number, party, region, timestamp, and updated voter status (hasVoted = true).

■ **Form / Format:** BSON binary document over TLS connection to MongoDB Atlas database.

■ **Method / Protocol:** MongoDB Wire Protocol with TLS 1.3 encryption across cluster replica sets.

■■ **Security Applied:** Atomic document updates (findOneAndUpdate). AES-256 storage encryption at rest. Audit log preserved in IST format.

STEP 13: Terminal Confirmation & Voter Feedback

■■ ~ 2.5 seconds (Visual & audio confirmation)

■■ **What Happens:** What Happens: Node.js returns HTTP 200 receipt to ESP32 with the transaction hash and block number. ESP32 triggers 3 short pips and 1 long confirmation tone on the buzzer, displays 'Vote Recorded! Secured on Web3' on the LCD for 2.5 seconds, clears local memory, and returns to standby scan mode.

■ **Data Transferred:** Receipt JSON with transaction hash and block number -> LCD display command -> Buzzer audio tone trigger.

■ **Form / Format:** JSON Receipt (~200B) -> I2C bus byte packet (0x27 address) -> GPIO 33 PWM audio tone.

■ **Method / Protocol:** Wi-Fi TCP/IP -> I2C Display Bus (100 kHz) -> Hardware PWM Active Buzzer chime.

■■ **Security Applied:** Immediate session termination: local voter tracking flags updated, memory buffers cleared, terminal returns to standby prompt.

MASTER DATA TRANSFER, SECURITY & TIMING MATRIX

Side-by-side comparison of every step in the VoteChain voting lifecycle:

Step # & Stage	Data Form / Format	Method / Protocol	Security Applied	Time
1. Optical Scan	Photons -> 256x288 Grayscale	TIR Optical CMOS	Anti-tamper Prism, Volatile RAM	**150 ms**
2. Minutiae DSP	512-Byte Feature Vector	AS608 Internal DSP	One-Way Minutiae, Zero Flash	**100 ms**
3. UART Transfer	4x 139B Binary Frames	UART Serial2 (57.6k)	16-Bit Packet Checksum	**90 ms**
4. Wi-Fi API Post	1024-Char Hex JSON	Wi-Fi HTTP /verify	WPA2-AES Encryption, Watchdog	**45 ms**
5. Engine Match	Probe vs N Minutiae Tables	Node.js In-Memory	Spatial/Angular Gates (dr<=18px)	**30 ms**
6. Double-Vote Preflight	SHA-256 Hash + JSON-RPC	Mongoose + Sepolia RPC	Zero-Knowledge Salted Hash	**90 ms**
7. Token Return	HMAC Auth Token JSON	HTTP 200 over Wi-Fi	HMAC-SHA256, 60s Expiry	**30 ms**
8. Touch Ballot	Capacitive GPIO Pulse	Hardware Interrupt	25s Timeout, Mutual Exclusion	**2.5 s**
9. Vote Dispatch	Party + Auth Token JSON	Wi-Fi HTTP /vote	Regional Franchise Boundary	**40 ms**
10. Anonymize & Sign	ABI castVote(Hash, Party)	Ethers.js v6 ECDSA	Secp256k1 Curve, Nonce Queue	**20 ms**
11. Sepolia Consensus	Raw Signed Tx Hex	JSON-RPC HTTPS / TLS	PoS Global Consensus, Immutable	**12 s (Block)**
12. Dual Storage	BSON Audit & Regional Doc	MongoDB Protocol TLS	AES-256 At Rest, Atomic Ops	**35 ms**
13. LCD Receipt	JSON Receipt + Audio PWM	I2C 0x27 + GPIO 33	Session Cleared, Immediate Reset	**2.5 s**

THE FIVE PILLARS OF VOTECHAIN SECURITY

1. Zero Sensor Flash Persistence: At no time is citizen biometric minutiae committed to the R307 sensor's non-volatile EEPROM memory. All capture, feature extraction, and transfer occur in volatile SRAM buffers that vanish immediately when power is cycled.
2. Point-to-Point Hardware Isolation: The serial channel between the optical sensor and ESP32 utilizes a dedicated physical UART bus guarded by 16-bit arithmetic checksums on every packet frame.
3. Decoupled Identity & One-Way Anonymization: Citizen Voter IDs are never broadcast to the public blockchain. A one-way salted SHA-256 digest decouples voter demographic records in MongoDB from the immutable vote cast on Ethereum Sepolia.
4. Single-Use Cryptographic Authorization: Every ballot requires a server-issued HMAC-SHA256 authorization token with a strict 60-second expiration. The token is revoked the millisecond the vote is cast, preventing replay and ballot stuffing.
5. Immutable Proof-of-Stake Consensus: Once the transaction is mined into an Ethereum block, it is cryptographically sealed by global validators. No electoral administrator, politician, or attacker can alter, delete, or re-order cast ballots.